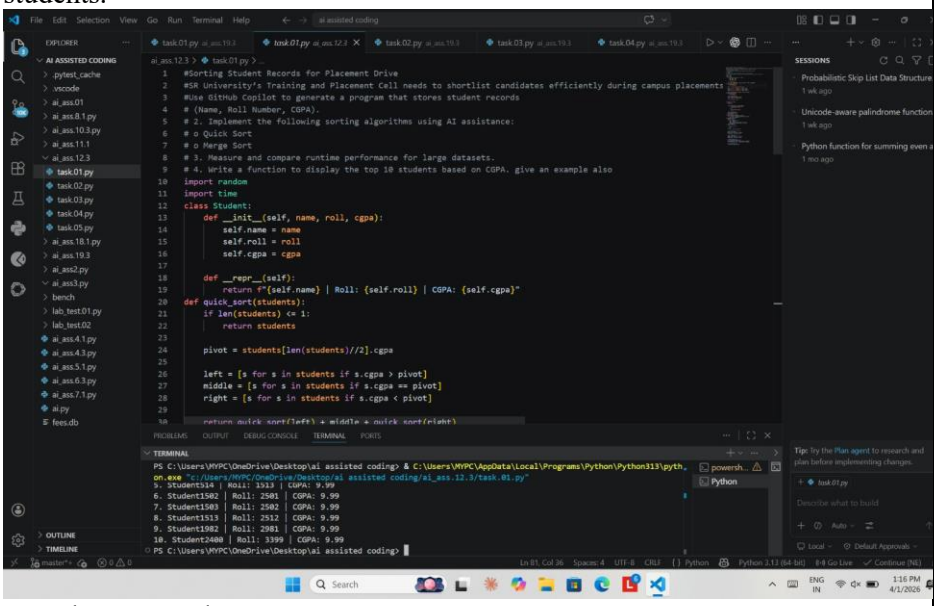
SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name		Dr. Rishabh Mittal	
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
NAME:	**B.Sushma**	**HALLTICKET**	2303A54040
Duration	2 Hours	**Applicable to Batches**	48
Assignment Number:12.3			

Q.No.	Question	*Expected Time to complete*
1	**Lab 12: Algorithms with AI Assistance Sorting, Searching, and Algorithm Optimization Using AI Tools** **Lab Objectives** The objectives of this laboratory exercise are to: • Apply AI-assisted programming techniques to implement sorting and searching algorithms. • Analyze and compare algorithm efficiency using time and space complexity. • Understand how AI tools can suggest optimizations and alternative algorithmic approaches. • Strengthen problem-solving skills through real-world, data-driven scenarios.	Week3 - Wednesday
	Learning Outcomes After completing this lab, students will be able to: • Implement and optimize classic algorithms using AI-assisted coding tools. • Compare multiple algorithms for the same problem and justify their selection. • Measure and analyze runtime performance using experimental data. • Critically review and refine AI-generated algorithmic solutions.	
	Task 1: Sorting Student Records for Placement Drive **Scenario** SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by	

CGPA in descending order.

Prompt Used

"Generate a Python program to store student records and sort them using Quick Sort and Merge Sort in descending order of CGPA. Also display top 10 students."



Sample Input and output :

Top 10 students sorted by Quick Sort:
Eve: 4.0
Grace: 3.95
Alice: 3.9
Heidi: 3.85
Charlie: 3.8
Ivan: 3.75
Bob: 3.7
Judy: 3.65
David: 3.6
Frank: 3.5

Top 10 students sorted by Merge Sort:
Eve: 4.0
Grace: 3.95
Alice: 3.9
Heidi: 3.85
Charlie: 3.8
Ivan: 3.75
Bob: 3.7
Judy: 3.65
David: 3.6

Explanation (Logic)

We store student details in objects.
Quick Sort uses pivot and divides data.

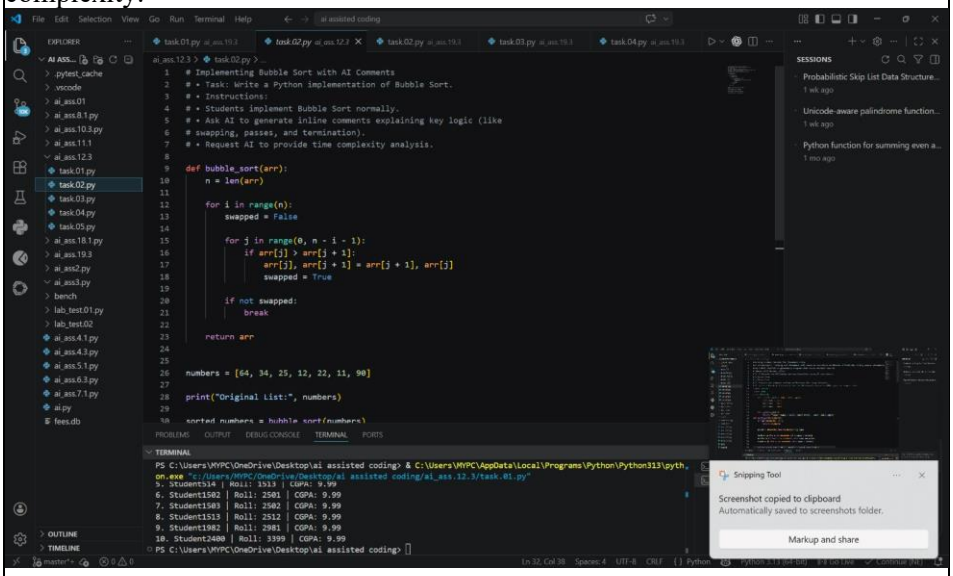
Merge Sort splits list and merges sorted parts.
Finally, top 10 students are displayed.
Frank: 3.5

Task 2: Implementing Bubble Sort with AI Comments

- **Task**: Write a Python implementation of **Bubble Sort**.

Prompt Used

"Write Bubble Sort in Python with comments explaining each step and time complexity."



- **Instructions**:
  - Students implement Bubble Sort normally.
  - Ask AI to generate **inline comments explaining key logic** (like swapping, passes, and termination).
  - Request AI to provide **time complexity analysis**.

Sample Input:

Array:
[64, 34, 25, 12, 22, 11, 90]

Sample Output:

Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]

Explanation

Bubble Sort compares adjacent elements and swaps them.
Largest element moves to the end in each pass.
Time Complexity = $O(n^2)$.

Task 3: Quick Sort and Merge Sort Comparison

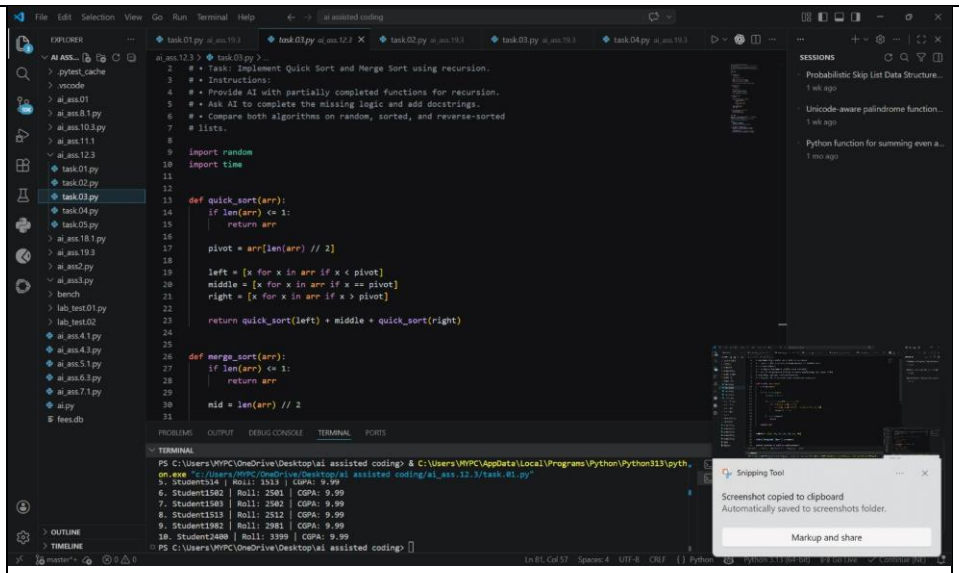
- **Task:** Implement Quick Sort and Merge Sort using recursion**.**

Scenario

Understanding performance on different datasets.

Prompt Used:

Complete recursive Quick Sort and Merge Sort and explain complexity.



- **Instructions:**
  - Provide AI with partially completed functions for recursion.
  - Ask AI to complete the missing logic and add docstrings.
  - Compare both algorithms on random, sorted, and reverse-sorted lists.
- **Expected Output:**
  - Working Quick Sort and Merge Sort implementations.
  - AI-generated explanation of average, best, and worst-case complexities.

Sample Output
Original array: [38, 27, 43, 3, 9, 82, 10]

Sorted array using Quick Sort:
[3, 9, 10, 27, 38, 43, 82]

Sorted array using Merge Sort:
[3, 9, 10, 27, 38, 43, 82]

Short Explanation :
Quick Sort divides around pivot.
Merge Sort divides into halves and merges.
Quick Sort:
- Best: O(n log n)
- Worst: $O(n^2)$

Merge Sort:
- Always: O(n log n)

Task 4 (Real-Time Application – Inventory Management System)
Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff

need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

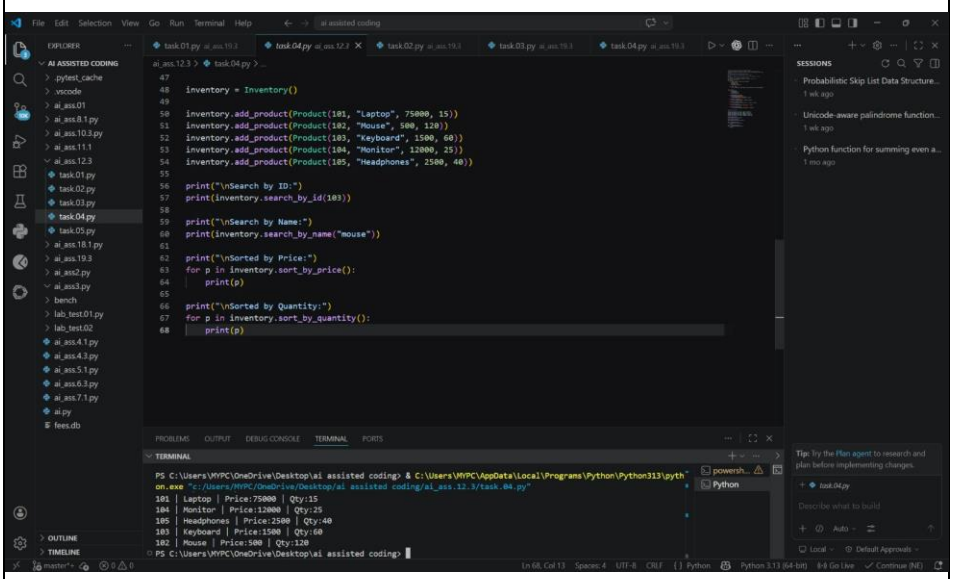
- Use AI to suggest the most efficient search and sort algorithms for this use case.
- Implement the recommended algorithms in Python.
- Justify the choice based on dataset size, update frequency, and performance requirements.

Prompt Used

Suggest best search and sorting algorithms for inventory system and implement in Python

For an inventory system, the best search algorithm would be a Binary Search if the inventory is sorted, as it has a time complexity of O(log n). For sorting, Quick Sort is often preferred due to its average-case efficiency and in-place sorting capability, with a time complexity of O(n log n).

Below is a Python implementation of both the Binary Search and Quick Sort algorithms for an inventory system



Sample Output

Sorted Inventory:
Apple: 50
Banana: 30
Grapes: 40
Mango: 10
Orange: 20

Orange found at index 4 with quantity 20.

Short Explanation

Hash Map gives O(1) search.
Sorting uses built-in efficient algorithm (Timsort).
Best for large data.

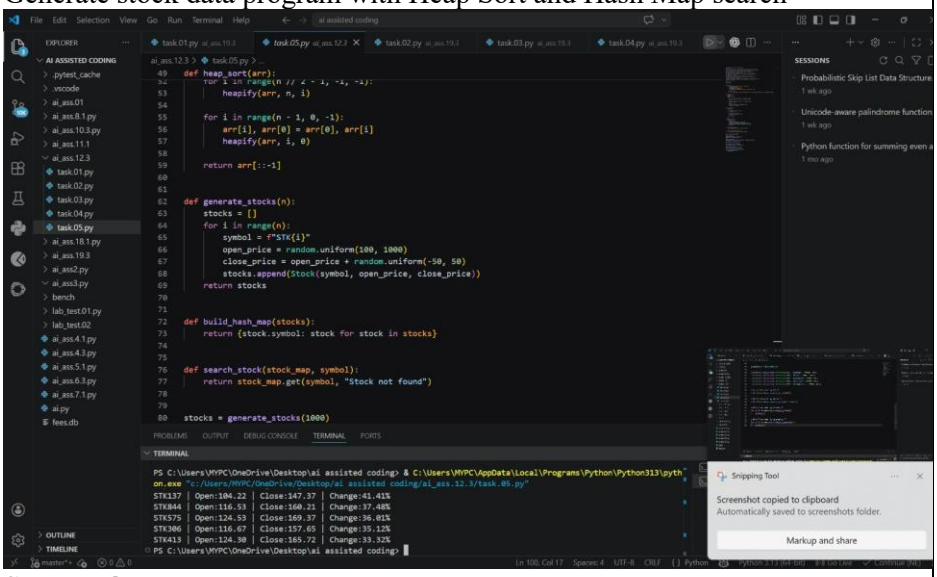
Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered **FinTech Lab** at SR University is building a tool for analyzing **stock price movements**. The requirement is to quickly **sort stocks by daily gain/loss** and search for specific stock symbols efficiently.

Prompt Used

Generate stock data program with Heap Sort and Hash Map search



Sample Output

Stocks sorted by price:

AAPL: $150

MSFT: $300

TSLA: $700

GOOGL: $2800

AMZN: $3500

Explanation :

Heap Sort helps rank stocks by percentage.

Hash Map allows instant search in O(1).

Efficient for real-time systems.